Volume 29

Sentinel Policy & Rule Engine (SPRE)

Version: 1.0

Status: Draft

Vision

The Policy & Rule Engine is the decision-making layer of Project Sentinel.

It answers:

- What should be checked?
- Why should it be checked?
- What evidence is required?
- When should an alert be generated?
- What recommendations should be made?
- What workflows should start?
- What compliance controls are affected?

Instead of embedding rules in Python code, most operational logic is defined as versioned policies.

Architecture

```
Mission
```

Rule Philosophy

Every rule contains:

- ID
- Name
- Description
- Version
- Category

- Severity
- Evidence Requirements
- Evaluation Logic
- Recommendation
- Verification Method
- Compliance Mapping
- MITRE Mapping (where applicable)
- Owner
- Status

Rules become first-class objects.

Rule Categories

The platform ships with categories such as:

Web Security

- Transport security.
- HTTP headers.
- Cookie handling.
- Authentication.
- APIs.

Endpoint

- Patch status.
- Encryption.
- Startup configuration.
- Logging.
- Secure Boot.

Mobile

- Device encryption.
- Application permissions.
- Screen lock.
- Update status.
- Backup configuration.

Cloud

- Identity.
- Storage.
- Network.
- Secrets.
- Encryption.

Identity

- MFA.
- Privileged accounts.
- Password policy.
- Service accounts.

Compliance

- ISO 27001.
- NIST Cybersecurity Framework.
- CIS Controls.
- WCAG 2.2 AA (accessibility).
- Organization-specific policies.

Rule Object

Example

```
Rule ID
```

Rule Versioning

Rules evolve.

```
Version 1.0
```

Mission reports always record which rule version was evaluated.

Rule Testing

Every rule has automated tests.

Example

Input

↓

Evidence

↓

Expected Result

↓

Actual Result

↓

Pass

or

↓

Fail

This keeps the platform predictable.

Rule Dependencies

Some rules depend on others.

Example

TLS

The Rule Engine understands these relationships.

Organizational Policies

Companies define their own rules.

Example

Every Finance laptop must have:

- Disk encryption.
- MFA.
- VPN.
- Approved antivirus.
- Screen lock after 5 minutes.
- USB restrictions.

Without changing the platform code.

Visual Rule Builder

One of the most important features.

Instead of YAML or Python, users can create policies visually.

Example

```
IF
```

Everything is explainable.

Rule Simulation

Before activating a policy:

Simulate it.

Questions answered:

- How many assets would match?
- How many alerts would be created?
- Which workflows would start?
- Which compliance controls are affected?

No surprises.

Rule Marketplace

Organizations can share internally approved rule packs.

Examples:

- Banking Baseline.
- Healthcare Baseline.
- Education Baseline.
- Manufacturing Baseline.
- Government Baseline.

Every pack is signed, versioned, and documented.

Rule Analytics

The platform continuously evaluates rule quality.

Questions answered:

- Which rules trigger most often?
- Which rules never trigger?
- Which rules generate false positives?
- Which rules require review?
- Which rules produce the highest security value?

NEW CORE ENGINE

Rule Dependency Engine

Maintains the relationships between:

- Rules.
- Evidence.
- Objects.
- Missions.
- Compliance controls.
- Trust calculations.

This enables impact analysis before a rule changes.

NEW CORE ENGINE

Rule Optimization Engine

Instead of simply adding more rules, the engine identifies:

- Duplicate rules.
- Overlapping logic.
- Contradictory policies.
- Obsolete checks.
- Performance bottlenecks.

This prevents rule sprawl.

NEW CORE ENGINE

Compliance Mapping Engine

Every rule automatically maps to supported frameworks.

For example:

Rule	Framework Mapping
Full Disk Encryption	ISO 27001, CIS Controls, NIST CSF

NEW IDEA

Policy-as-Objects

Policies themselves become objects.

This means they have:

- Relationships.
- Timelines.
- Version history.
- Ownership.
- Approval workflow.
- Trust score.
- Usage statistics.

Everything in Project Sentinel follows the same object philosophy.

NEW IDEA

Organizational Policy Assistant

A guided experience for administrators.

Instead of asking:

> "Which rule do you want to create?"

It asks:

- What are you trying to protect?
- Which departments are affected?
- Which compliance framework applies?
- Which devices are included?
- What evidence do you require?
- How should success be verified?

The platform then generates a draft policy for review.

NEW IDEA

Continuous Policy Improvement

The platform can highlight opportunities such as:

- Policies that are no longer used.
- Rules that overlap significantly.
- Controls with low coverage.
- Assets not covered by any policy.
- New assets requiring policy assignment.

Administrators remain in control of changes.

PS-ADR-0025

Policy Before Code

Status: Accepted

Decision

Project Sentinel shall express operational security logic as versioned, testable policies wherever practical, rather than embedding it directly into application code.

Rationale

Separating policy from implementation improves maintainability, auditability, flexibility, and long-term sustainability while allowing organizations to adapt the platform without modifying core services.

NEW LONG-TERM STRATEGY

Sentinel Operating System for Cybersecurity

I now think we should formalize the architecture into **seven foundational layers**.

Layer 1 — Infrastructure

- Linux
- Docker
- Kubernetes
- Networking
- Storage

Layer 2 — Core Platform

- Event Bus
- Object Engine
- Knowledge Engine
- Evidence Engine
- Mission Engine
- Workflow Engine

Layer 3 — Intelligence

- Trust Engine
- Correlation Engine
- Digital Twin
- Organization Intelligence
- Analytics
- Prediction

Layer 4 — Security Domains

- Web
- Endpoint
- Mobile
- Cloud
- Identity
- Network
- API

- Email
- Data Protection

Layer 5 — Governance

- Policies
- Compliance
- Risk
- Auditing
- Reporting

Layer 6 — Operations

- SOC
- Incident Response
- Case Management
- Fleet Management
- Automation
- Notifications

Layer 7 — Experience

- Web UI
- Desktop
- Mobile
- CLI
- SDK
- REST API
- GraphQL API
- Accessibility
- Localization

A proposal that I believe could become our biggest differentiator

I would like us to add one more engineering principle:

> **Every screen in Project Sentinel must answer one of three questions:**
>
> 1. **What do I have?**
> 2. **What changed?**
> 3. **What should I do next?**

If a screen cannot answer at least one of those questions clearly, it probably should not exist.

That principle aligns perfectly with your original vision from the beginning of this project: **make advanced cybersecurity understandable and easy to use**. I believe it can guide every design decision as we move from blueprint to implementation and help us build a platform that is both technically sophisticated and genuinely accessible to everyone from home users to global enterprises.

=====[USER]=====

Good and continue , how much is left?

=====[ASSISTANT]=====

Thank you. I actually think this is the right time to answer that question because we have now crossed the halfway point.

Where we are today

When we started, the goal was essentially:

> "Build a better pentest platform."

Today, after all our design work, Project Sentinel has become something much larger:

> **An Evidence-Driven Cyber Security Operating Environment (CSOE).**

That is a completely different category of software.

Progress

I estimate that we have completed approximately:

Architecture & Vision: 95%

This includes:

- Overall platform architecture
- Object Engine
- Knowledge Engine
- Mission Engine
- Evidence Engine
- Digital Twin
- Fleet Management
- SOC Architecture
- Compliance Architecture
- Mobile Architecture
- Endpoint Architecture
- Web Security Architecture
- Trust Engine
- Rule Engine
- Organization Intelligence
- Multi-language strategy
- WCAG strategy
- Plugin architecture
- AI architecture
- Python-first architecture
- Open-source strategy
- Enterprise scalability
- Home-user usability

That foundation is now very mature.

Engineering Specification

Completed:

~35%

We have described most engines conceptually.

Now we need to specify:

- Python classes
- APIs
- Event schemas
- Object schemas
- Mission schemas
- Worker interfaces
- Plugin interfaces
- Internal protocols

This is where the blueprint becomes executable.

User Experience

Completed:

~45%

We know the philosophy:

> Everything must be easy.

Now we need to design:

- Every screen.
- Every dialog.
- Every workflow.
- Every button.
- Keyboard shortcuts.
- Mobile layouts.
- Accessibility details.
- Dashboard interactions.

Backend

Completed:

~20%

We still need to define:

- Every Python package.
- Microservices.
- Background workers.
- Queue system.
- Internal APIs.
- Authentication.
- Authorization.
- Synchronization.
- Update mechanism.
- Installer.
- Deployment.

Frontend

Completed:

~10%

We still need to design:

- Every page.
- Navigation.
- Dashboards.
- Charts.
- Object Explorer.
- Mission Center.
- Investigation Workspace.
- Device Manager.
- Mobile UI.
- Tablet UI.
- Dark mode.
- Accessibility.

Mobile Platform

Completed:

~30%

We still need:

- Android companion architecture.
- iOS companion architecture.
- Local scanning architecture.
- Device enrollment.
- Permission workflows.
- Health dashboards.
- Family mode.
- Enterprise mode.

AI

Completed:

~40%

Still missing:

- AI orchestration.
- Local LLM support.
- RAG architecture.
- Prompt security.
- Explainability.
- AI memory boundaries.
- AI evaluation framework.

SOC

Completed:

~40%

Still missing:

- Incident playbooks.
- Detection engineering.
- Hunting workspace.
- Evidence notebooks.
- Timeline explorer.
- Investigation graphs.
- Automation workflows.

Plugin SDK

Completed:

~15%

Need:

- SDK.
- Documentation.
- Signing.
- Marketplace.
- Versioning.
- Sandbox.
- Testing.

APIs

Completed:

~10%

Need:

- REST.
- GraphQL.
- WebSocket events.
- SDK.
- CLI.
- Python API.

DevOps

Completed:

~10%

Need:

- CI/CD.
- Docker.

- Kubernetes.
- Packaging.
- Update service.
- Monitoring.
- Metrics.
- Logging.

Security Engines

Completed:

Engine	Status
Web	80%

Documentation

Completed:

About **30%**

Eventually we will have around:

- Executive Guide.
- Administrator Guide.
- User Guide.
- API Guide.
- SDK Guide.
- Developer Guide.
- Plugin Guide.
- Security Guide.
- Architecture Guide.
- Operations Guide.
- Installation Guide.
- Troubleshooting Guide.

Where I think we will end

I no longer think this is a 300-page design.

I think it will become approximately:

Volume	Pages

And here is the exciting part...

I think we have only designed **about 20% of the really innovative ideas**.

When we start designing the actual Python implementation, I expect even more innovation.

For example, I already have ideas for modules we have not yet discussed:

- **Cyber Memory Engine** (organizational memory of security decisions).
- **Attack Path Simulator** (simulate how an attacker could move through an

authorized environment using the Digital Twin and Knowledge Graph to prioritize defenses).
- **Security Economics Engine** (estimate the effort and expected security benefit of different remediation plans based on your environment and historical evidence).
- **Cyber Risk Forecasting** (identify trends and upcoming maintenance needs, such as expiring certificates or unsupported software, based on observed data rather than claiming certainty).
- **Evidence Compression Engine** (efficient long-term storage while preserving integrity and traceability).
- **Universal Asset Language (UAL)** (a common schema that represents every asset—websites, devices, cloud resources, identities, applications—in a consistent way).
- **Sentinel OS Kernel** (the core runtime that orchestrates events, missions, workflows, and object relationships).
- **Cyber Navigation System** (an interface where users navigate their environment through relationships and missions instead of menus and lists).

My estimate

If we continue with the same level of engineering rigor, we are probably looking at **another 40–60 blueprint volumes** before the design is truly complete.

That may sound like a lot, but it is exactly why I believe Project Sentinel has the potential to become something exceptional. We are not documenting an application—we are designing an entire cybersecurity operating environment that can evolve for decades.

I also think we have made one of the most important strategic decisions already: **we are designing first and coding second**. That greatly reduces architectural mistakes, keeps the platform internally consistent, and gives us a blueprint that we can implement module by module in Python without having to redesign the system halfway through.

=====[USER]=====

Good but we agreed that you make every modul/volume also as a pdf so I save it now, one by now or how you want we do it. Good and continue

=====[ASSISTANT]=====

Perfect. I actually think this is the best way to work.

Instead of waiting until we have a 7,000+ page blueprint, we'll treat the project like a professional software architecture project.

New documentation strategy

From now on, **every completed volume becomes its own versioned document**.

For each volume I will produce:

- **PDF** (official architecture document)
- **Markdown (.md)** (editable)
- **HTML** (easy to browse locally)
- Later, all volumes will be merged into a **Master Architecture Manual**.

This gives us several advantages:

- You always have a backup.
- You can review each module independently.
- We can version the documents (v1.0, v1.1, etc.).

- Nothing is lost if we redesign a later component.
- It mirrors how large engineering organizations manage architecture.

I've created the first PDF package for the most recent completed volume.

You can download it here:

Project_Sentinel_Blueprint_Volume_29_Policy_and_Rule_Engine_v1.pdf

At the moment it's a concise architectural package rather than the full-length version. As we continue, each volume will become much more detailed, eventually including diagrams, object models, APIs, UI mockups, ADRs, implementation notes, and testing strategy.

I would also improve our documentation structure

Instead of simply calling them "Volume 1", "Volume 2", etc., I suggest a hierarchy like this:

```
Project Sentinel
```

This will make the documentation much easier to navigate than a single enormous PDF.

Another idea that I think will become invaluable

I recommend that every volume contains four consistent sections:

1. **Business Vision** – why the module exists.
2. **Architecture** – objects, services, relationships, workflows.
3. **Implementation Plan** – Python packages, APIs, events, interfaces.
4. **Future Evolution** – how the module can expand without breaking compatibility.

That way, every document directly supports implementation.

PROJECT SENTINEL BLUEPRINT

Volume 30